

Scaling CustomersDot VMs

Overview

This runbook documents the process for scaling CustomersDot VMs both horizontally (adding new VMs) and vertically (resizing existing VMs). This capability is critical for handling increased load, particularly for Usage Billing and other high-traffic scenarios.

Prerequisites

Required Access

- **SRE Access:** Standard SRE permissions for config-mgmt repository
- **InfraSec Approval:** Required for Teleport token creation (for horizontal scaling)
- **Fulfillment Team Access:** Maintainer access to the following repositories:
 - customersdot-ansible
 - customers-gitlab-com

Key Repositories

- **Infrastructure:** config-mgmt
- **Provisioning:** customersdot-ansible
- **Application:** customers-gitlab-com

Important Notes

- CustomersDot VMs run on Ubuntu 20.04 LTS with a specific boot image
- All VMs must be registered with Teleport for SSH access. This requirement covers both human administrative access and automated deployment and provisioning processes.
- The provisioning and deployment process requires coordination between SRE and Fulfillment teams

Horizontal Scaling (Adding New VMs)

Step 1: Create Infrastructure MR

Create a merge request in config-mgmt that:

1. Adds new VM(s) to the node map in the appropriate environment (**stgsub** or **prdsb**)
2. Creates Teleport provisioning tokens for the new VM(s)

Example MR: config-mgmt!12567

Key configuration points:

- Use the node map structure to define individual VMs
- Specify the correct `os_boot_image_override` (currently `ubuntu-os-cloud/ubuntu-2004-focal-v20240`)
- Include Teleport token creation in the same MR
- Ensure proper zone distribution for high availability

Approval Requirements:

- SRE code owner approval
- InfraSec approval (for Teleport tokens)

Step 2: Monitor VM Creation

Once the MR is merged and Atlantis applies the changes:

1. Monitor the VM startup via serial console:

```
gcloud compute --project=<PROJECT_ID> instances tail-serial-port-output <VM_NAME> --zone=<ZONE>
```

Example for staging:

```
gcloud compute --project=gitlab-subscriptions-staging instances tail-serial-port-output <VM_NAME> --zone=<ZONE>
```

2. Wait for the startup script to complete (typically 5-10 minutes)
3. Look for the message: `Startup finished in X.XXXs (kernel) + Xmin X.XXXs (userspace)`

Note: With the Teleport token pre-created, Teleport should automatically work once the VM is up.

Step 3: Provision the VM

Run the Ansible provisioning job from the customersdot-ansible repository.

The ability to run these CI pipelines is limited to maintainers. In case of emergency, the EOC can use their admin account to kick off the pipelines.

Manual provisioning steps:

1. Navigate to the customersdot-ansible project
2. Go to CI/CD → Pipelines → Run Pipeline
3. Select the appropriate branch (usually `master`)
4. Run the pipeline and wait for the provision job to complete

Reference: Manual provisioning documentation

Expected duration: 25 - 30 minutes

Step 4: Deploy the Application

Run the deployment job from the customers-gitlab-com repository.

The ability to run these CI pipelines is limited to maintainers. In case of emergency, the EOC can use their admin account to kick off the pipelines.

Manual deployment steps:

1. Navigate to the customers-gitlab-com project
2. Go to CI/CD → Pipelines
3. Find the latest pipeline for the staging branch
4. Manually trigger the **deploy-staging** or **deploy-production** job

Reference: Manual deployment documentation

Expected duration: 5-10 minutes

Step 6: Verify the VM is Serving Traffic

Monitor the CustomersDot Overview Dashboard to confirm:

1. The new VM appears in the metrics
2. The VM is receiving traffic
3. No errors are being reported
4. Response times are normal

Step 7: Remove the machine from teleport tokens

1. Create an MR to remove the machine name from `environments/teleport-production/tokens.tf` (added in step 1).

Vertical Scaling (Resizing Existing VMs)

Vertical scaling involves changing the machine type of existing VMs to increase or decrease resources.

Important Notes

- VMs must be stopped to change machine type
- This causes downtime for the specific VM being resized
- Resize VMs one at a time to maintain service availability
- Total time per VM: approximately 2-5 minutes

Step 1: Resize the VM

For each VM you want to resize:

```
# Remove from target pools
```

```
gcloud compute target-pools remove-instances prdsub-tcp-lb-customers-http --instances=custom
```

```
gcloud compute target-pools remove-instances prdsub-tcp-lb-customers-https --instances=custo
```

```
# wait 5 mins or monitor active connections
```

```
# Stop the VM
```

```
gcloud compute instances stop <VM_NAME> --zone=<ZONE> --project=<PROJECT_ID>
```

```

# Change the machine type
gcloud compute instances set-machine-type <VM_NAME> --machine-type=<NEW_MACHINE_TYPE> --zone=

# Start the VM
gcloud compute instances start <VM_NAME> --zone=<ZONE> --project=<PROJECT_ID>

# Wait for the VM to fully boot
while ! gcloud compute ssh <VM_NAME> --zone=<ZONE> --command="uptime" --ssh-flag="-o Connect

# Add machine back to target pools
gcloud compute target-pools add-instances prdsub-tcp-lb-customers-http --instances=customers
gcloud compute target-pools add-instances prdsub-tcp-lb-customers-https --instances=customer

```

Example (resizing to n1-standard-8):

```

gcloud compute target-pools remove-instances stgsub-tcp-lb-customers-http --instances=customers
gcloud compute target-pools remove-instances stgsub-tcp-lb-customers-https --instances=customers
gcloud --project gitlab-subscriptions-staging compute instances stop customers-03-inf-stgsub
gcloud --project gitlab-subscriptions-staging compute instances set-machine-type customers-03-inf-stgsub
gcloud --project gitlab-subscriptions-staging compute instances start customers-03-inf-stgsub
gcloud compute target-pools add-instances stgsub-tcp-lb-customers-http --instances=customers
gcloud compute target-pools add-instances stgsub-tcp-lb-customers-https --instances=customers

```

Step 2: Update Terraform Configuration

Once all VMs have been resized, update the Terraform configuration to match:

1. Create an MR in config-mgmt
2. Update the `machine_type` in the node map for each resized VM
3. Run `atlantis plan -- --refresh` to verify it's a no-op plan
4. Get the MR approved and merged

Example MR: config-mgmt!12571

Important: The Terraform plan should show no changes if the VMs were resized correctly.

Troubleshooting

Cannot SSH to New VM

Symptoms: Unable to connect via `tsh ssh` to a newly created VM

Solutions:

1. Verify Teleport token was created in the config-mgmt MR
2. Check the serial console for Teleport registration errors
3. Ensure the VM has fully booted (check for “Startup finished” message)
4. Verify your Okta user ID matches your Chef user ID

Break-glass procedure: If Teleport is completely unavailable, see the break-glass SSH access documentation

Chef Fails with Ubuntu Advantage Error

Symptoms: Chef run fails with error about `esm-infra` or Ubuntu Advantage

Background: This is a known issue with the Ubuntu Advantage cookbook on Ubuntu 20.04

Solutions:

1. This has been addressed in recent Chef cookbook updates
2. If it persists, see `platform/runway/team#715`
3. The issue should not occur on Ubuntu 22.04 (planned upgrade)

Provisioning Fails with `psycopg2` Error

Symptoms: Ansible provisioning fails with “Failed to import the required Python library (`psycopg2`)”

Solution: This should be fixed in the Ansible playbooks, but if it occurs:

```
# SSH to the VM
tsh ssh <VM_NAME>

# Install the package manually
sudo apt-get update
sudo apt-get install -y python3-psycopg2

# Retry the provisioning job
```

Provisioning Fails with `nginx` Error

Symptoms: Ansible fails with “No such file or directory: ‘`nginx`’”

Solution: This should be fixed in the Ansible playbooks to use `/usr/sbin/nginx`, but if it occurs, ensure the Ansible playbooks are up to date.

VM Not Receiving Traffic After Deployment

Checklist:

1. Verify the `pet_name=customers` label is set
2. Check the CustomersDot dashboard
3. Verify the VM is in the correct instance group
4. Check `nginx` is running: `sudo systemctl status nginx`
5. Check application logs: `/home/customersdot/CustomersDot/current/log/production.log`

Access and Permissions

Who Can Perform These Operations?

Horizontal Scaling:

- **Infrastructure MR:** Any SRE (requires SRE + InfraSec approval)
- **Provisioning:** Maintainers on customersdot-ansible (currently limited SREs + Fulfillment team) or SRE with admin account.
- **Deployment:** Maintainers on customers-gitlab-com (Fulfillment team + limited SREs) or SRE with admin account.

Vertical Scaling:

- **VM Resize:** Any SRE with GCP access to the CustomersDot projects
- **Terraform Update:** Any SRE (requires SRE approval)

Current SRE Maintainers on Fulfillment Repositories

As of November 2024:

- Pierre Jambet
- Cameron McFarland
- Gonzalo Servat

Note: Additional SREs may need maintainer access for emergency scaling scenarios. File an Access Request if needed.

Restricted Access Note

Direct SSH access to CustomersDot VMs is restricted due to audit requirements. Not all SREs have the GCP IAM permissions to use `gcloud compute ssh` with IAP. Teleport is the primary access method.

Timeline Expectations

Horizontal Scaling (Adding One VM)

Step	Duration	Notes
Create and approve MR	30-60 min	Depends on reviewer availability
VM creation and boot	5-10 min	Automated via Terraform
Provisioning	25-30 min	May need retry if transient failures
Deployment	5-10 min	Usually succeeds first try
Total	75-110 min	Assuming no issues

Vertical Scaling (Resizing One VM)

Step	Duration	Notes
Stop, resize, start VM	2-5 min	Per VM
Update Terraform	15-30 min	MR creation and approval
Total per VM	17-35 min	Do VMs sequentially

Related Documentation

- CustomersDot Overview
- CustomersDot Ansible Documentation
- Fulfillment Escalation Process
- Infrastructure Change Management

Reference Issues and MRs

- Original discovery and testing: production-engineering#27880
- Node map conversion (staging): config-mgmt!12504
- Node map conversion (production): config-mgmt!12530
- Example horizontal scaling MR: config-mgmt!12567
- Example vertical scaling MR: config-mgmt!12571